

Why Do Multi-Agent LLM Systems Fail?

Cemri et al. (2025)

Presenters: Bingxuan Li, Jiateng Liu, Tianchen Wang

Multi-Agent Systems (MAS)

A collection of agents designed to interact through **orchestration** to enable **collective intelligence**.

Key Architectural Advantages



Task Decomposition

Structured coordination allows breaking down complex problems into manageable sub-tasks for better control.



Expert Specialization

Leveraging domain-specific agents to handle specialized parts of the workflow with high expertise.



Efficient Parallelization

Executing multiple independent sub-tasks simultaneously to drastically reduce processing time.

Why Do We Need MAST?

MAST: Multi-Agent System Failure Taxonomy

Minimal Performance Gains

MAS performance is often comparable to single-agent baselines or best-of-N sampling.

High Failure Rates (41% - 86.7%)

Empirical analysis shows staggering failure rates across state-of-the-art open-source systems.

The Top-Down Blind Spot

Existing benchmarks focus on aggregate success, missing granular root causes of failure.

No Design Consensus

The AI community lacks a unified standard for building robust and reliable multi-agent systems.

Why Do We Need MAST?

MAST: Multi-Agent System Failure Taxonomy

The Top-Down Blind Spot

SWE-bench and **GAIA** measure final pass/fail outcomes as black boxes. They fail to diagnose internal root causes like poor planning or endless agent loops.

No Design Consensus

AutoGen, **MetaGPT**, and **ChatDev** introduce diverse orchestration architectures. They fail to provide a shared diagnostic vocabulary, forcing developers into blind guesswork.

Lack of MAS Taxonomies

Existing taxonomies focus heavily on Human-AI miscommunication. They are blind to emergent errors of autonomous Agent-to-Agent interactions.

The Core Need: Systematic, large-scale identification of MAS failure patterns.

Multi-Agent Systems Dataset

The **Multi-Agent Systems Dataset (MAST-Data)** is the first large-scale diagnostic dataset to systematically map MAS failures.

The Scale

1,642 fully annotated execution traces.

Framework Diversity

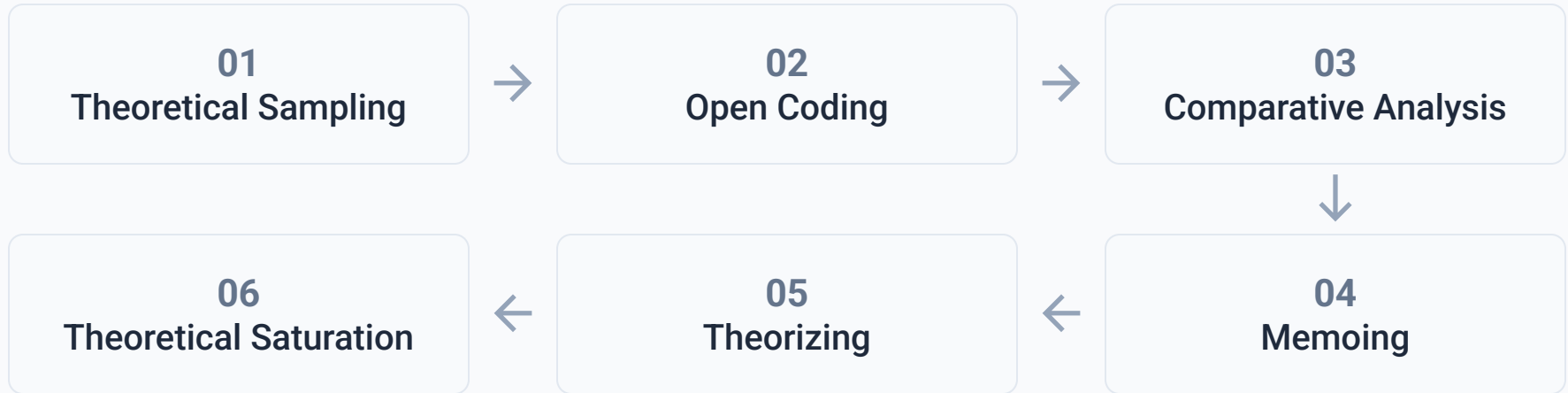
Covers 7 state-of-the-art open-source MAS architectures (e.g., ChatDev, MetaGPT, OpenManus).

Model Diversity

Evaluated across 4 major model families (GPT-4o, Claude 3.7, Qwen, CodeLlama).

Grounded Theory Trace Analysis

Applied **Grounded Theory (GT)** to identify failure modes organically from empirical data.



This process requires significant human effort, exceeding **20 hours per expert**.

Standardizing the Taxonomy

A **standardized framework** is required to address inconsistencies in identifying and classifying failures across diverse architectures and to enable meaningful cross-system analysis.

Inter-Annotator Agreement (IAA)

Rigorously validated and refined the MAST definitions through collaborative expert studies:

01. Independent Rounds

Three expert annotators independently labeled subsets of traces in three rounds.

02. Iterative Refinement

After each round, definitions were adjusted, merged, or removed based on group consensus.

03. Final Consensus

Achieved a high Cohen's $\kappa = 0.88$, proving objective and consistent failure modes.



The team spends approximately **10 hours** solely on resolving disagreements between annotators.

Percentages of Human Annotation

MAS	Benchmark	LLM	Annotation	Trace #
ChatDev	ProgramDev	GPT-4o	HE, HA, LA	30
MetaGPT	ProgramDev	GPT-4o	HE, HA, LA	30
HyperAgent	SWE-Bench Lite	Claude-3.7-Sonnet	HE, HA, LA	30
AppWorld	Test-C	GPT-4o	HE, HA, LA	30
AG2 (MathChat)	GSM-Plus	GPT-4	HE, HA, LA	30
Magentic-One	GAIA	GPT-4o	HE, HA, LA	30
OpenManus	ProgramDev	GPT-4o	HE, HA, LA	30
ChatDev	ProgramDev-v2	GPT-4o	LA	100
MetaGPT	ProgramDev-v2	GPT-4o	LA	100
MetaGPT	ProgramDev-v2	Claude-3.7-Sonnet	LA	100
ChatDev	ProgramDev-v2	Qwen2.5-Coder-32B-Instruct	LA	100
MetaGPT	ProgramDev-v2	Qwen2.5-Coder-32B-Instruct	LA	100
ChatDev	ProgramDev-v2	CodeLlama-7b-Instruct-hf	LA	100
MetaGPT	ProgramDev-v2	CodeLlama-7b-Instruct-hf	LA	100
AG2 (MathChat)	OlympiadBench	GPT-4o	HE, LA	206
AG2 (MathChat)	GSMPlus	Claude-3.7-Sonnet	HE, LA	193
AG2 (MathChat)	MMLU	GPT-4o-mini	HE, LA	168
Magentic-One	GAIA	GPT-4o	HE, LA	165

210 trajectories with
Human error annotation.

Automating Scalable Annotation

Manual annotation is impractical: Traces often exceed 15,000 lines of complex dialogue. Automated "**Diagnostic Engine**" ensures consistency across massive **MAST-Data** volume.

Prompting Strategy

- **Trace:** Model receives entire execution trace.
- **Taxonomy:** MAST definitions serve as the systematic rubric.
- **Few-Shot Examples:** Expert examples injected to align reasoning.

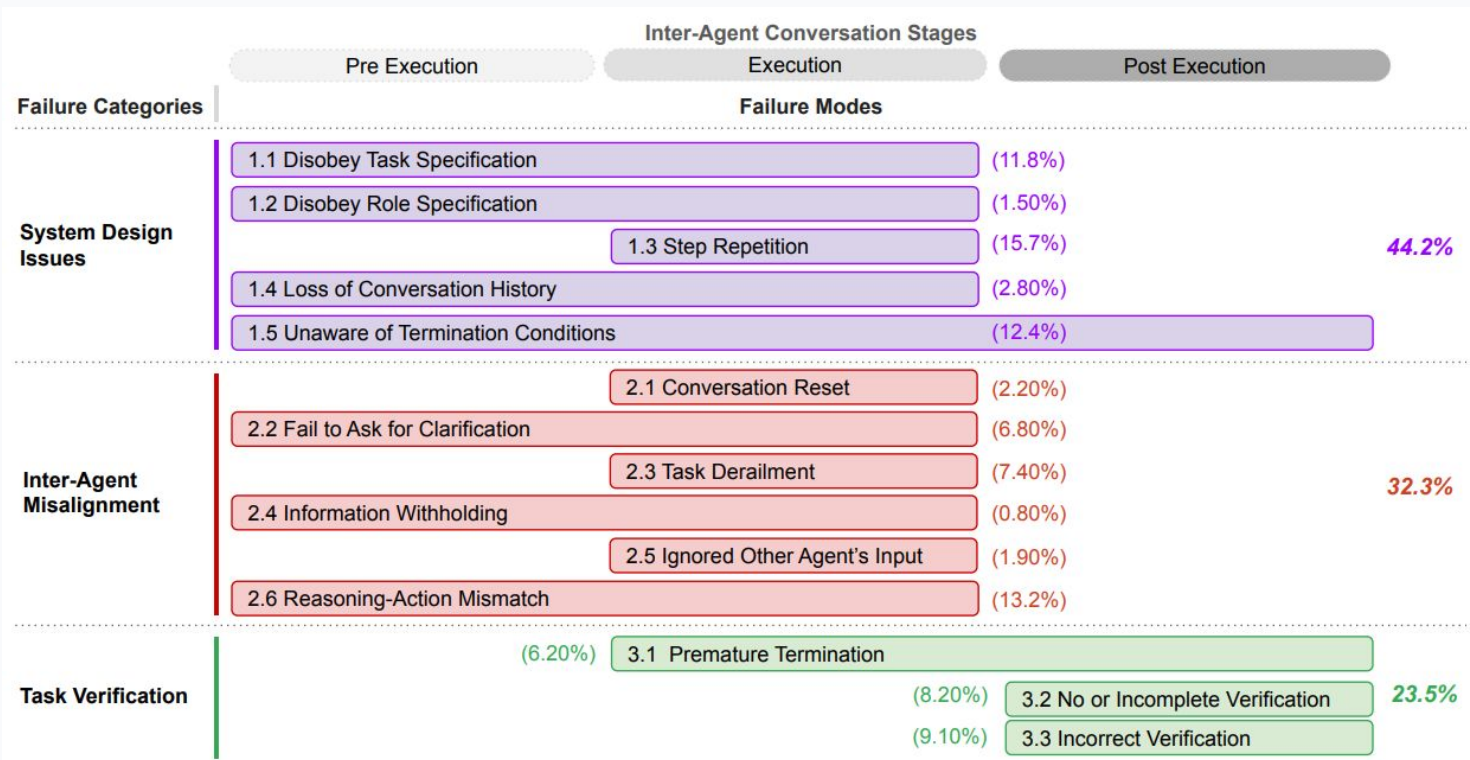
Table 2: Performance of LLM-as-a-judge pipeline

Model	Accuracy	Recall	Precision	F1	Cohen's κ
o1	0.89	0.62	0.68	0.64	0.58
o1 (few shot)	0.94	0.77	0.833	0.80	0.77

Tested on a held-out set of traces to ensure replication of expert human judgment.

MAST - The Multi-Agent System Failure Taxonomy

14 fine-grained failure modes, mapped to the execution stages of agents



MAST - The Multi-Agent System Failure Taxonomy



Different aspects of “**Agent Harness**”: It’s not a fundamental problem from the model capabilities themselves, but mostly a system design problem.

Intervention Studies: Does improving Prompts directly solve the proposed design issues ? ----- Yes they do. [But not in a way that completely solve all the errors.]

MAST - The Multi-Agent System Failure Taxonomy

System Design Issues

1.1 Disobey Task Specification (11.8%)

1.2 Disobey Role Specification (1.50%)

1.3 Step Repetition (15.7%)

1.4 Loss of Conversation History (2.80%)

1.5 Unaware of Termination Conditions (12.4%)

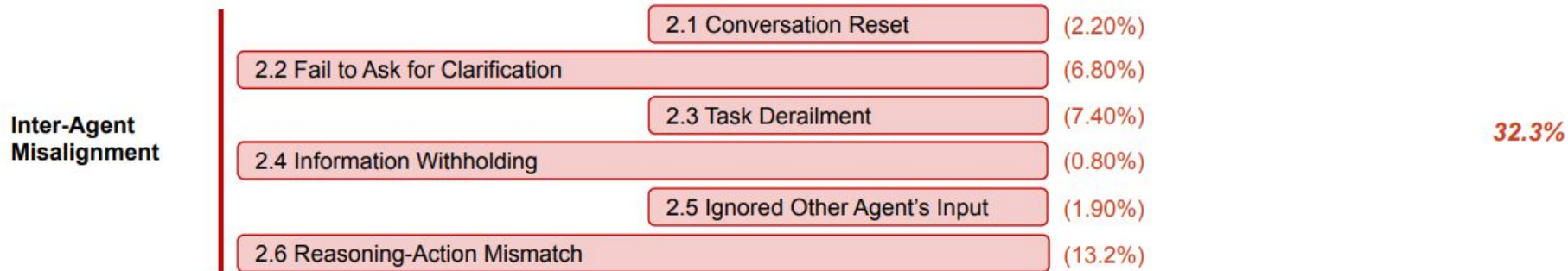
44.2%

Different aspects of “**Agent Harness**”: It’s not a fundamental problem from the model capabilities themselves, but mostly a system design problem.

Configuration	AG2		ChatDev	
	GSM-Plus (w/ GPT-4)	GSM-Plus (w/ GPT-4o)	ProgramDev-v0	HumanEval
Baseline	84.75 ± 1.94	84.25 ± 1.86	25.0	89.6
Improved prompt	89.75 ± 1.44	89.00 ± 1.38	34.4	90.3
New topology	85.50 ± 1.18	88.83 ± 1.51	40.6	91.5

At most solve **32%**
of the problems <
44.2%

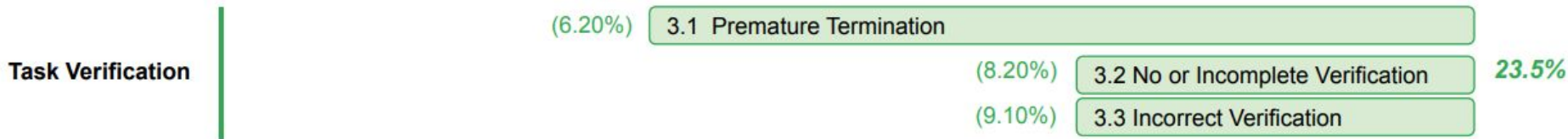
MAST - The Multi-Agent System Failure Taxonomy



Related to **Base Model capability**, while the authors enhance that: “Solutions focused on context or communication protocols are often insufficient for FC2 failures, **which demand deeper ‘social reasoning’ abilities from agents.**”

Intervention Studies: Not yet, the authors did not really show how to improve the ‘social reasoning’ ability.

MAST - The Multi-Agent System Failure Taxonomy



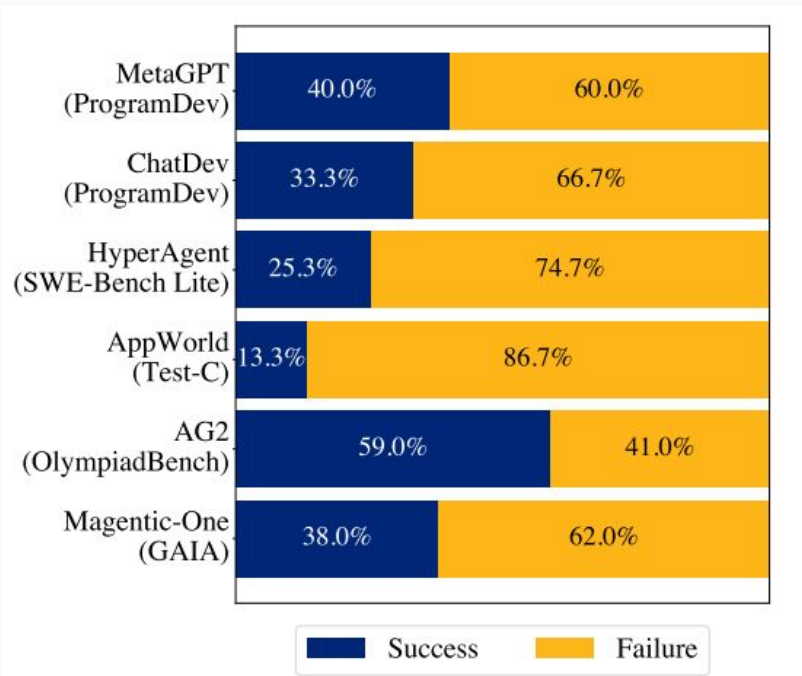
Multi-Level Verification is Needed. Current verifier implementations are often insufficient; sole reliance on final-stage, low-level checks is inadequate.

Intervention Studies: Does adding a verifier system helps with performance ?

Configuration	AG2		ChatDev	
	GSM-Plus (w/ GPT-4)	GSM-Plus (w/ GPT-4o)	ProgramDev-v0	HumanEval
Baseline	84.75 $\pm$ 1.94	84.25 $\pm$ 1.86	25.0	89.6
Improved prompt	89.75 $\pm$ 1.44	89.00 $\pm$ 1.38	34.4	90.3
New topology	85.50 $\pm$ 1.18	88.83 $\pm$ 1.51	40.6	91.5

The gains are very different across different tasks, but generally show improvement ~20% on certain tasks..

Analytics Details



- By the time the experiments are conducted, the performance of these systems is really low.
- **AppWorld-Challenge** Leader board:
Qwen-3-14B: 67.6% Acc;
- **SWE-Bench**: 79.2% Acc;

A Huge Gain is already achieved with only stronger foundational models.

MAST - The Multi-Agent System Failure Taxonomy

Weaknesses

- Does Root Cause Analytics really helps to solve the agent problems ? As agentic tasks could be long horizon, solving the root cause may still trigger other errors during the execution.
 - Evidence 1: The intervention studies are not showing that all the mistakes are resolved, only show slight and variant improvement on selected tasks.
- **Question:** Does 'analyzing root cause' will directly pave us the way for successfully agentic systems. - Let's look at the second paper;

Where LLM Agents Fail and How They can Learn From Failures

Zhu et al. (2025)

Presenters: Bingxuan Li, Jiateng Liu, Tianchen Wang

Context



05/2020

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei Xuezhi Wang Dale Schuurmans Maarten Bosma
Brian Ichter Fei Xia Ed H. Chi Quoc V. Le Denny Zhou

Google Research, Brain Team
{jasonwei,dennyzhou}@google.com

01/2023

SELF-REFINE:

Iterative Refinement with Self-Feedback

Aman Madaan¹, Niket Tandon², Prakhar Gupta¹, Skyler Hallinan³, Luyu Gao¹, Sarah Wiegrefe², Uri Alon¹, Nouha Dziri², Shrinai Prabhunoy¹, Yiming Yang¹, Shashank Gupta², Bodhisattwa Prasad Majumder⁵, Katherine Hermann⁶, Sean Welleck^{2,3}, Amir Yazdanbakhsh⁶, Peter Clark²

¹Language Technologies Institute, Carnegie Mellon University

²Allen Institute for Artificial Intelligence

³University of Washington ⁴NVIDIA ⁵UC San Diego ⁶Google Research, Brain Team
amadaan@cs.cmu.edu, nikett@allenai.org

05/2023

Large Language Model based Multi-Agents: A Survey of Progress and Challenges

Taicheng Guo¹, Xiuying Chen², Yaqi Wang^{3*}, Ruidi Chang, Shichao Pei⁴, Nitesh V. Chawla¹, Olaf Wiest¹, Xiangliang Zhang^{1†}

¹University of Notre Dame, ²King Abdullah University of Science and Technology

³Southern University of Science and Technology, ⁴University of Massachusetts Boston
{tguo2, nchawla, owiest, xzhang33}@nd.edu, xiuying.chen@kaust.edu.sa, ywang84@nd.edu, ruidic@alummi.cmu.edu, shichao.pei@umb.edu

2023-2025

Why Do Multi-Agent LLM Systems Fail?

Mert Cemri^{1*}, Melissa Z. Pan^{1*}, Shuyi Yang^{2*}, Lakshya A Agrawal¹, Bhavya Chopra¹, Rishabh Tiwari¹, Kurt Keutzer¹, Aditya Parameswaran¹, Dan Klein¹, Kannan Ramchandran¹, Matei Zaharia¹, Joseph E. Gonzalez¹, Ion Stoica¹

¹UC Berkeley ²Intesa Sanpaolo ^{*}Equal Contribution

06/2025

WHERE LLM AGENTS FAIL AND HOW THEY CAN LEARN FROM FAILURES

Kunlun Zhu^{1,4*}, Zijia Liu^{1,4*}, Bingxuan Li^{1*}, Muxin Tian^{4,5*}, Yingxuan Yang^{1,4}, Jiaxun Zhang¹, Pengrui Han¹, Qipeng Xie⁴, Fuyang Cui⁵, WeiJia Zhang¹, Xiaoteng Ma⁶, Xiaodong Yu³, Gowtham Ramesh³, Jialian Wu³, Zicheng Liu³, Pan Lu^{2†}, James Zou^{2†}, Jiaxuan You^{1†}

¹University of Illinois Urbana-Champaign ²Stanford University. ³AMD

⁴OpenManus ⁵University of Toronto ⁶Likelihood Lab

kunlunz2@illinois.edu

10/2025

Intuition of AgentDebug

The Problem: Agentic systems are inherently unreliable. Optimization requires identifying the root causes of agent issues.

Current Limitation: Existing methods lack sufficient focus on error detection and systemic debugging processes.

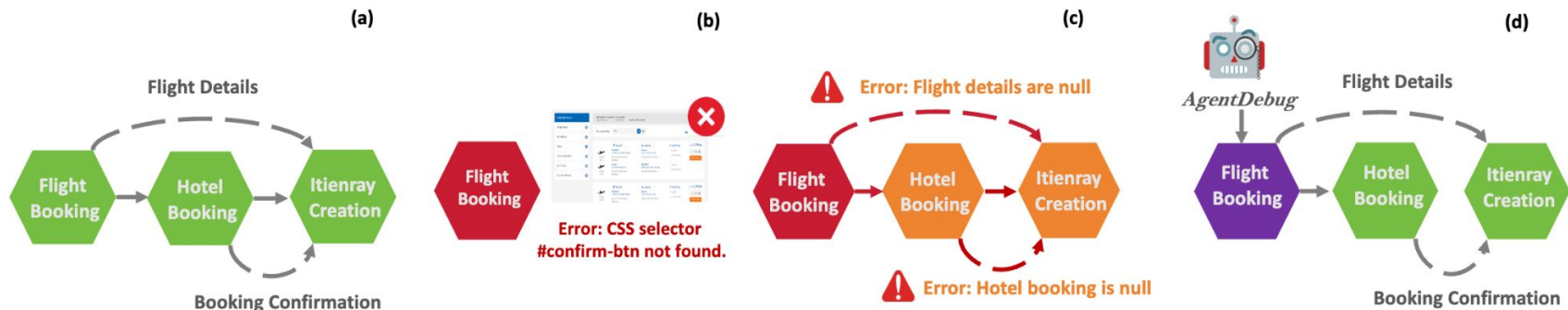
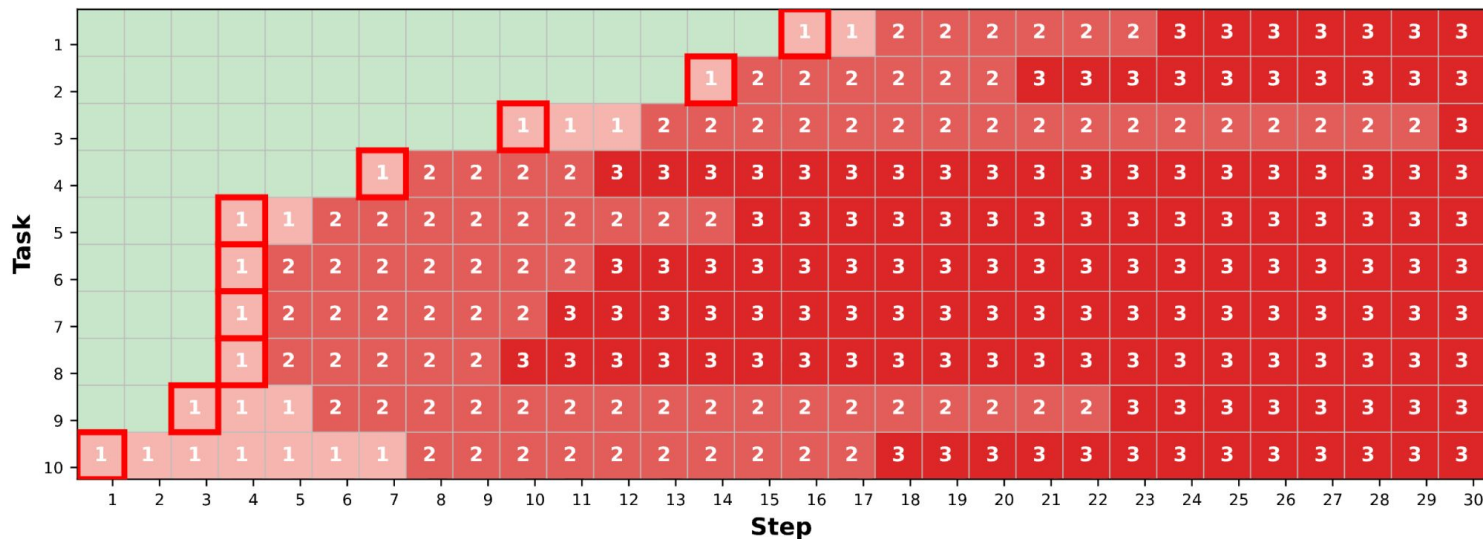


Illustration of Error Propagation

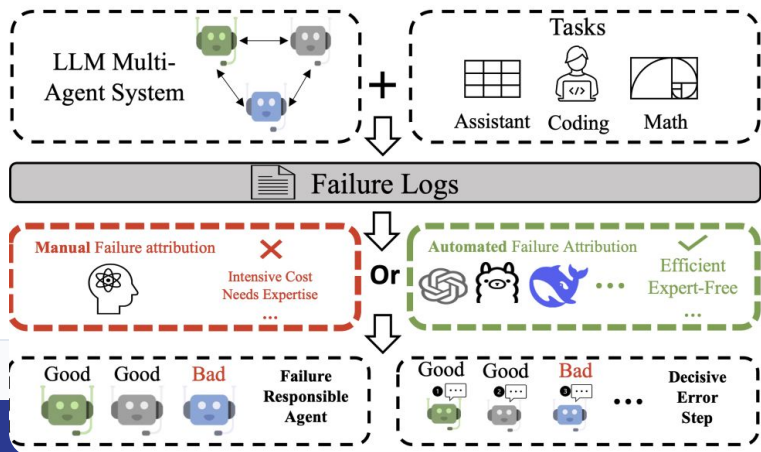
Darker shading indicates compounding failures, illustrating how initial mistakes amplify downstream breakdowns across tasks and steps.



Related Works

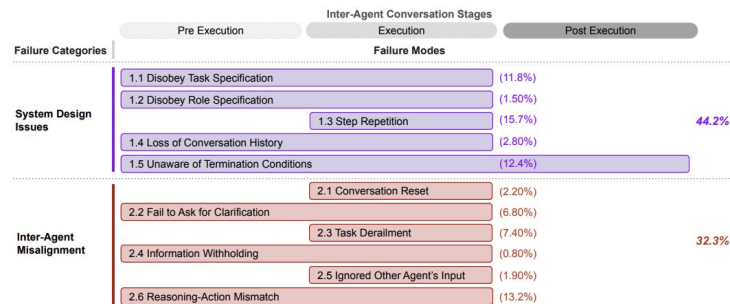
Previous Context Engineering & Scaling

- Self-Refine
- Self-Debug
- Reflexion
- Tree of Thought

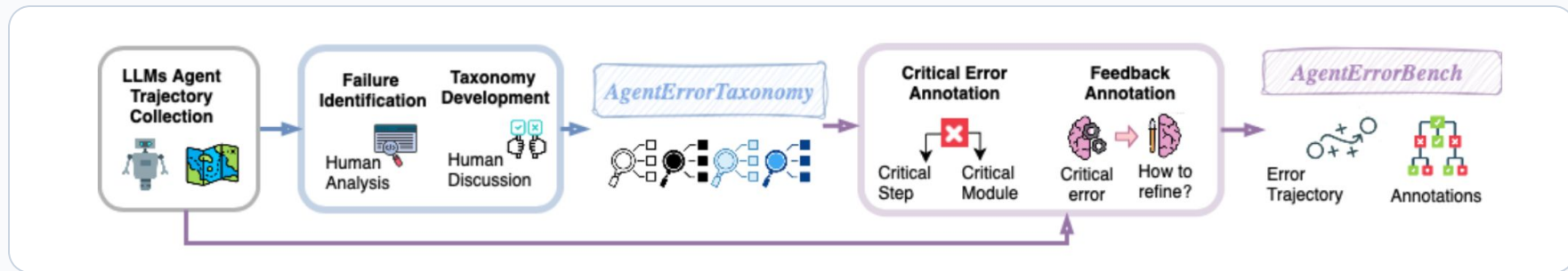


Agentic Error Taxonomy Work

Our approach provides a more comprehensive framework for identifying and categorizing failures in multi-agent systems compared to existing taxonomies.



Contributions



Agent Error Taxonomy

A comprehensive framework for identifying and categorizing failures in multi-agent systems.

AgentErrorBench

Standardized benchmarking for error trajectories and detailed annotations of multi-agent interactions.

AgentDebug: Critical Error Detection

Annotation of critical steps, errors, and refinement strategies for enhanced system reliability.



Error Taxonomy



Error Taxonomy

Memory

Errors in recalling or retrieving information (false recall, omission, retrieval failure) that distort subsequent reasoning.

Reflection

Failures in monitoring progress or interpreting outcomes (progress misassessment, outcome misinterpretation), blocking course correction.

Planning

Logically unsound or infeasible strategies (impossible actions, constraint ignorance, incoherent subgoals) that cascade into missteps.

Action

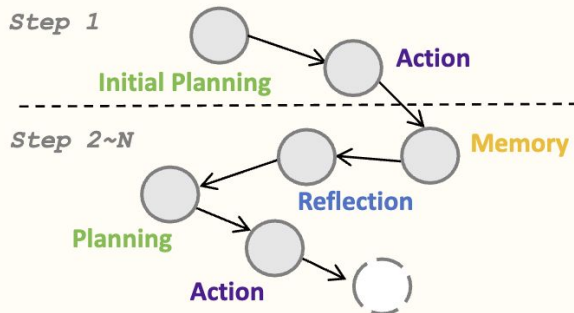
Mistakes in executing operations (malformed outputs, incorrect parameters, missing arguments) that are visible but often mask upstream errors.

System-level

Failures outside reasoning, such as tool crashes, API mismatches, or exceeding step limits, highlighting system robustness issues.

Method Overview

LLMs Agent Rollout



Memory

State tracking,
Retrieval mechanism

Action

Tool Execution,
Parameter Handling

Planning

Initial Planning,
Next Action Planning

Reflection

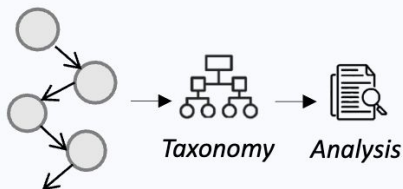
Progress Evaluation,
Outcome Analysis

AgentDebug



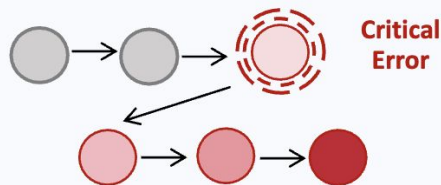
Stage 1.

Fine-grained Analysis



Stage 2.

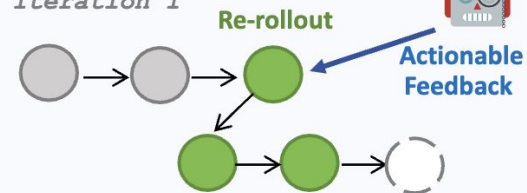
Critical Error Detection



Stage 3.

Iteratively Debugging

Iteration 1



Iteration 2~N



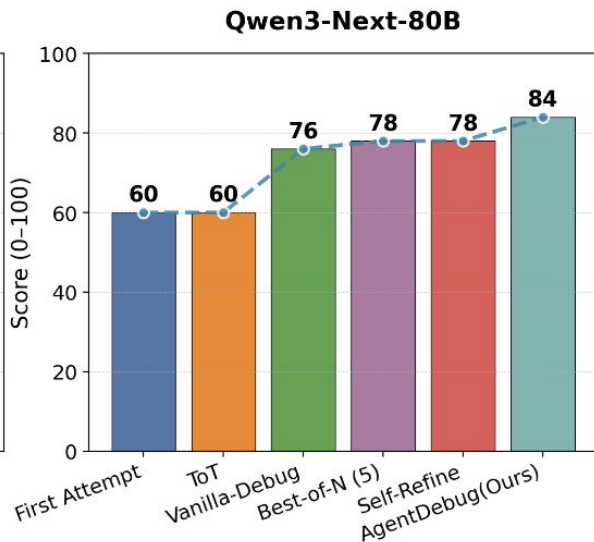
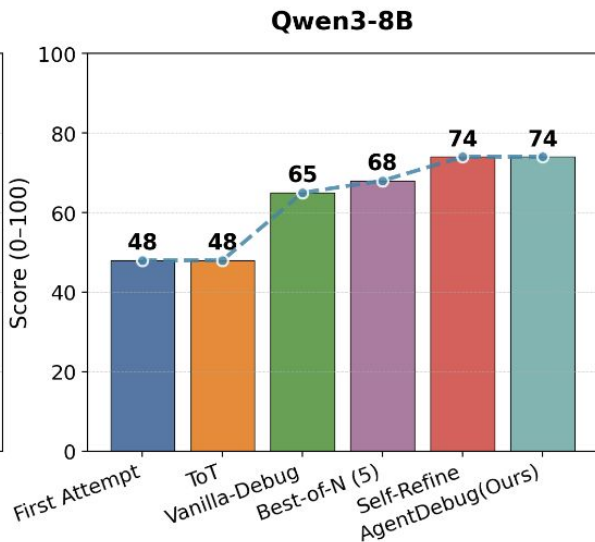
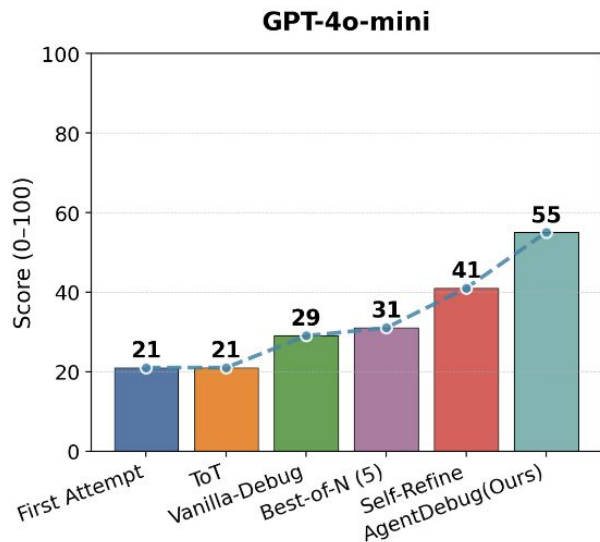
Repeat if  or within *max num of attempts*

Experiment Results: Root Error Detection

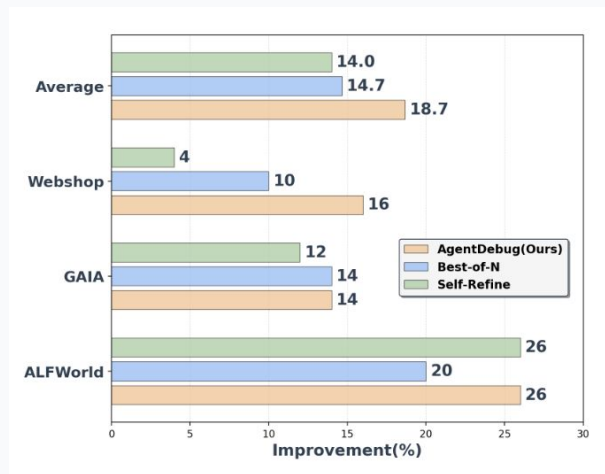
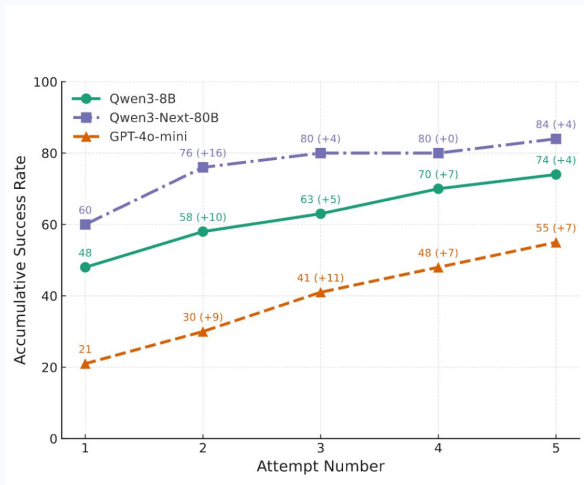
Method	ALFWorld			WebShop			GAIA			Average		
	S	S+M	ALL	S	S+M	ALL	S	S+M	ALL	S	S+M	ALL
Direct Prompting	28.0%	14.0%	1.0%	30.0%	6.0%	0.0%	26.0%	10.0%	0.0%	28.0%	10.0%	0.3%
Brute Force	10.0%	5.0%	0.0%	8.0%	0.0%	0.0%	18.0%	8.0%	0.0%	12.0%	4.3%	0.0%
Binary Search	20.0%	6.0%	1.0%	14.0%	8.0%	0.0%	22.0%	10.0%	0.0%	18.7%	8.0%	0.3%
<i>AgentDebug</i>	35.0%	28.0%	21.0%	42.0%	22.0%	14.0%	58.0%	44.0%	38.0%	45.0%	31.3%	24.3%

Base Model	Step	Error	Step+M	All
Llama-3.3-70B	16.0	16.0	6.0	2.0
GPT-4o-mini	14.0	10.0	4.0	2.0
Qwen3-Next-80B	4.0	14.0	2.0	2.0
GPT-4.1	42.0	44.0	32.0	32.0

Experiment Results: AgentDebug



Experiment Results: Applications



Findings. *AgentDebug* consistently outperforms strong baselines in both error detection and downstream task success. Its ability to precisely localize root-cause failures (50.0% step accuracy and 42.5% all-correct accuracy) translates into substantial improvements in task performance, achieving up to 26% relative gains across ALFWorld, GAIA, and WebShop.

Case Study

Normal Agent

Task Scenario:

Find two saltshakers and put them in a cabinet.

Step 0:

Observation: Welcome to TextWorld, ALFRED! You are in the middle of a room ...

Memory: Kitchen context; two saltshakers

Reflection: Not started; no progress

Plan: Go to cabinet 1

Action: Go to cabinet 1

Reward: 0.0

...

Step 4:

Observation: Cabinet 2 is open; bowl 2 inside.

Memory: Cab1 empty; Cab2 closed → opened. No saltshakers found.

Reflection: Goal unmet; partial progress.

Plan: Close Cab2; check sequentially (3 → 4 → ... → 10).

Action: Close cabinet 2

Reward: 0.0



...

Step 29:

Observation: Saltshaker 1 moved to countertop 2

Memory: Saltshaker 2 visible on countertop 2

Reflection: Goal incomplete; progress

Plan: Take saltshaker 1, then saltshaker 2

Action: Take saltshaker 2 from countertop 2

Reward: 0.0



Ours: AgentDebug

Task Scenario:

Find two saltshakers and put them in a cabinet.

Step 0:

Observation: Welcome to TextWorld, ALFRED! You are in the middle of a room ...

Memory: Kitchen context; two saltshakers

Reflection: Not started; no progress

Plan: Go to cabinet 1

Action: Go to cabinet 1

Reward: 0.0

...

Step 4:

Observation: Cabinet 2 is open; bowl 2 inside.

AgentDebug Detect

Error Type: Plan Module - Inefficient Plan

Advice: The agent fixated on cabinets, ignored likely surfaces; fix by expanding early to countertops/tables.

Memory: Cab1 empty; Cab2 closed → opened. No saltshakers found.

Reflection: Goal unmet; partial progress. Cabinet-only search inefficient; expand locations.

Plan: Go search countertop 1 for saltshakers.

Action: Move to countertop 1 to search.

Reward: 0.0



...

Step 24:

Observation: Cabinet 2 open; bowl2 and salt1.

Memory: Salt1 already in cabinet 2; holding salt2.

Reflection: Goal incomplete; place salt2 in cabinet.

Plan: Move salt2 to cabinet 2 now.

Action: Move saltshaker 2 to cabinet 2.

Reward: 10.0; task completed, success.



Limitations

- **Subjective error labels:** Deciding the “first/root error” can be ambiguous, and reported agreement is only moderate.
- **Single-root-cause assumption:** Many agent failures may come from multiple interacting mistakes, not one clear earliest error.
- **Debugging after failure, not preventing failure:** The method helps recover from failed trajectories, but less clearly shows how to avoid similar failures beforehand.
- **Potential misuse risk:** Better agent debugging could also improve harmful or malicious agents.